

Below is **clean, minimal, copy-paste-ready syntax** for every STL container / asset we discussed.

No fluff. Just what you actually type in high-level DSA code.

vector

```
cpp

vector<int> v;
vector<int> v(n);
vector<int> v(n, 0);

v.push_back(10);
v.pop_back();

v[i];
v.at(i);

v.size();
v.empty();

for (int x : v) { }
```

string

```
cpp

string s;
string s = "hello";

s.push_back('a');
s.pop_back();

s.size();
s.substr(l, len);

s[i];
```

unordered_map

```
cpp

unordered_map<int, int> mp;
```

```
mp[key] = value;
mp[key]++;

mp.find(key);
mp.count(key);

for (auto &p : mp) {
    p.first; // key
    p.second; // value
}
```

map

```
cpp

map<int, int> mp;

mp[key] = value;

mp.find(key);
mp.begin();
mp.end();

for (auto &p : mp) {
    p.first;
    p.second;
}
```

unordered_set

```
cpp

unordered_set<int> st;

st.insert(x);
st.erase(x);

st.find(x);
st.count(x);

st.size();
```

stack

cpp

```
stack<int> st;
```

```
st.push(x);
```

```
st.pop();
```

```
st.top();
```

```
st.empty();
```

queue

cpp

```
queue<int> q;
```

```
q.push(x);
```

```
q.pop();
```

```
q.front();
```

```
q.back();
```

```
q.empty();
```

deque

cpp

```
deque<int> dq;
```

```
dq.push_back(x);
```

```
dq.push_front(x);
```

```
dq.pop_back();
```

```
dq.pop_front();
```

```
dq.front();
```

```
dq.back();
```

priority_queue (max heap)

cpp

```
priority_queue<int> pq;
```

```
pq.push(x);  
pq.pop();  
  
pq.top();
```

priority_queue (min heap)

```
cpp  
  
priority_queue<int, vector<int>, greater<int>> pq;
```

pair

```
cpp  
  
pair<int, int> p;  
pair<int, int> p = {a, b};  
  
p.first;  
p.second;
```

tuple

```
cpp  
  
tuple<int, int, int> t;  
t = make_tuple(a, b, c);  
  
get<0>(t);  
get<1>(t);  
get<2>(t);
```

DSA pro defaults (memorize this)

```
cpp  
  
vector<int>&  
const vector<int>&  
unordered_map<int,int>  
priority_queue<int>  
deque<int>
```

If you can type these **without thinking**, STL will never slow you down again — it becomes muscle memory, not a decision.